

Running an LLM As a Local Server in LM Studio

Overview

You have experimented with using an LLM in LM Studio in chat mode. You did that by going to the Chat page, loading an LLM, creating a folder, and asking the LLM for information about some subject.

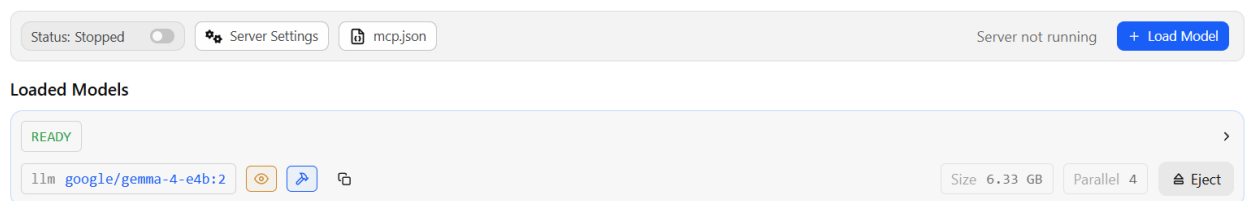
LM Studio also allows us to load the model and run it as a server on your local PC. Once we run it as a server, we can interact with it programmatically via code.

Running LLM as a Local Server

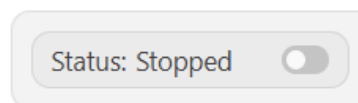
1. Go to the **Developer** page in LM Studio by clicking the toolbar Developer button:



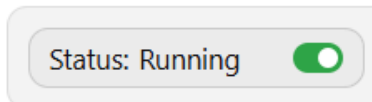
2. Under Loaded Models, if you don't see a model that is loaded, then click the blue **Load Model** button. In the snapshot below, it shows that I already have a model loaded: gemma-4-e4b.



3. In the upper left corner in the above snapshot, you see the status of the server. It indicates that it is Stopped.



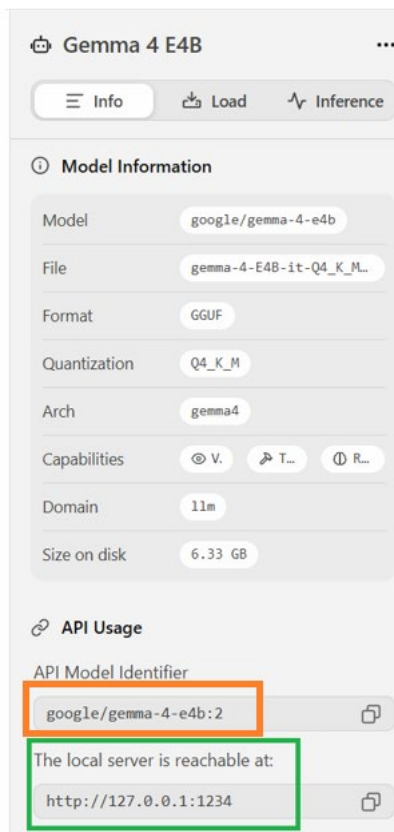
4. To start the server, toggle the button. Its status changes to Running and appears green



If you also look at the **Developer logs** tab at the bottom of the page you should see a log message that the server has started:

```
2026-08-02 14:44:42 [INFO] [LM STUDIO SERVER] -> POST http://localhost:1234/v1/embeddings
2026-08-02 14:44:42 [INFO]
2026-08-02 14:44:42 [INFO] [LM STUDIO SERVER] Logs are saved into C:\Users\alfred.nehme\lmstudio\server-logs
2026-08-02 14:44:42 [INFO] Server started.
2026-08-02 14:44:42 [INFO] Just-in-time model loading active.
```

On the right-hand side of the page, you should see a panel with information about the loaded model:



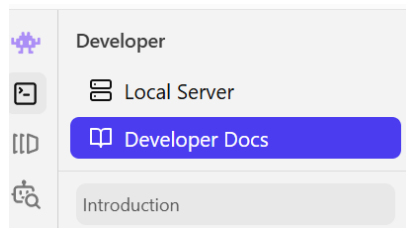
Highlighted in green is the URL of the server. Notice the IP address **127.0.0.1**. This IP address is a reserved special IP address that is equivalent to **localhost**. This

means the server is running locally. When you use <http://127.0.0.1> (or its <http://localhost> equivalent), the request is not forwarded to the Internet. Instead, it is handled by the operating system and looped back into the same machine. The 1234 number is the port on which it is listening.

Highlighted in orange is the LLM identifier. You will need this identifier when you write code to interact with the LLM.

Interacting with the LLM Server via Code

As you interact with the LLM Server, you will often need to look at the **LM Studio Developer Documentation**. You access it from the **Developer Docs** under the **Developer** page in LM Studio.



There are several ways to interact with the LLM Server, such as from TypeScript SDK, Python SDK, REST API, etc. In our case we are interested in Python. The Python SDK is `lmstudio-python` (documentation can be found [here](#)).

Complete the following to test that you can interact with your LLM Server:

1. Create a project folder named `LLMServerTest`.
2. Open `LLMServerTest` with VSCode.
3. Create a python virtual environment (like how you did in document “Python for VSCode”).
4. From the Terminal pane in VSCode, install the `lmstudio-python` SDK by installing package `lmstudio`:

```
pip install lmstudio
```

(In VSCode, if the Terminal pane is not visible, you can always open it by clicking

menu **View** → **Terminal**).

5. Under **LLMSERVERTEST** in VSCode, create a file named `check_server.py`.
6. Enter the following code in `check_server.py`:

```
# We need to use the package lmstudio in our code. So, we must import it
import lmstudio as lms

# Get a list of downloaded models
downloaded_models = lms.list_downloaded_models()

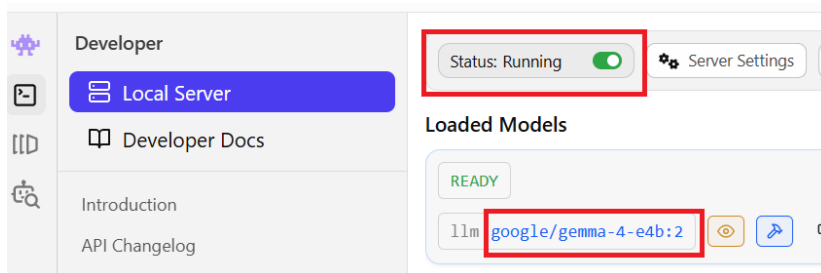
print("Downloaded Models:")
print("-----:")
for model in downloaded_models: # In python, we can loop through the items of a list using the for loop
    print(model.model_key)

print() # Adds an empty line for clarity
print("Model info:")
print("-----:")
# Get a handle to one of my models
model = lms.llm("google/gemma-4-e4b")
print(model.get_info().display_name + ": (size = " + str(float(model.get_info().size_bytes) / 1000000000) + " GB)")
```

In the code, `model.get_info().size_bytes` returns the model size in bytes. We divide by 1000000000 to get the size in gigabytes (1 GB = 10^9 bytes).

7. Click menu **File** → **Save all** to save the changes you just made.

Before you run the above code, verify that in LM Studio, your LLM is loaded and currently running as a server (you do that in the **Developer** page of LM Studio):



8. Run the code of `check_server.py` by clicking the **Run Python File** button.

If you get back a list of the models you have downloaded and basic info about the model you specified in the `llm()` function, then you have demonstrated that you can interact with the LLM running as a server on your computer. In my case, I see

the following output:

```
Downloaded Models:
-----:
google/gemma-4-e4b
text-embedding-nomic-embed-text-v1.5

Model info:
-----
Gemma 4 E4B: (6.326938154 GB)
```

I didn't get an output like the above. I got an error.

If you get an error that says something like “**LMStudioServerError: Model get/load error**”, then it is likely that your model is not loaded or the identifier you passed to the `llm()` function is wrong. In this case, go to LM Studio, navigate to the **Developer** page, and load the LLM and copy its identifier and make sure it is running (see steps 2 to 4 in the previous pages).

What is the difference between a model I get in the code such as the above and when I use AI in an application like ChatGPT?

When you use an application like ChatGPT, you are using a web interface (a website page) or a mobile app to interact with the model behind ChatGPT (e.g., GPT-5.6) and that is hosted in the cloud (not on your computer). So, GPT-x.y is the model (where x.y represents some version), and ChatGPT is the user application that allows you to interact with the model; It gives you a Textbox so that you can type questions and get back answers. These models are proprietary and owned by large companies such as OpenAI, Microsoft, Anthropic, Google, etc.

In the case we have here we also have a model (named **gemma-4-e4b**). But instead of it being hosted somewhere in the cloud, it exists on your computer and is exposed as a server. This way we can interact with it programmatically using a language like python. Obviously, this gives us power to do more things than just a Textbox to pose a question and get an answer back.

Question to think about and answer yourself:

→ Pair up with a student sitting next to you and answer the following question (**without searching the Internet or asking AI**).

On a piece of paper, write down 3 advantages you can think of when running an LLM locally instead of being hosted by some company somewhere in the cloud.